# 600

# HARNESS ENGINEERING

Building blocks for long-running agents

30 CATEGORIES | 600 ENTRIES | SOURCE LINKS

A practical catalog of design patterns, components and checks. Each entry includes a compact label and a plain-language explanation.

This guide expands the harness engineering theme of the @beamnxw article. Labels and summaries are authored for this collection; the reference links cover the underlying concepts.

Read the original article

```
> context / tools / control / state / evals
> 600 entries / 30 categories
> save progress. verify results. reuse the workflow.
```

# Contents

30 categories. Each chapter contains 20 entries.


01 **Task contracts** 001-020 03

02 **Project instructions** 021-040 04

03 **Context retrieval** 041-060 05

04 **Memory** 061-080 06

05 **Reusable skills** 081-100 07

06 **Planning** 101-120 08

07 **Workflow orchestration** 121-140 09

08 **The agent loop** 141-160 10

09 **Tool schemas** 161-180 11

10 **Tool execution** 181-200 12

11 **MCP & connectors** 201-220 13

12 **Permissions** 221-240 14

13 **Lifecycle hooks** 241-260 15

14 **Isolation & sandboxes** 261-280 16

15 **Specialized roles** 281-300 17

16 **Delegation** 301-320 18

17 **Models & reasoning effort** 321-340 19

18 **Goals & completion** 341-360 20

19 **Runtime state & checkpoints** 361-380 21

20 **Recovery & resuming** 381-400 22

21 **Validation & tests** 401-420 23

22 **Evidence & citations** 421-440 24

23 **Evaluation datasets** 441-460 25

24 **Graders & scoring** 461-480 26

25 **Observability** 481-500 27

26 **Resources & budgets** 501-520 28

27 **Environment & bootstrap** 521-540 29

28 **Deliverables & artifacts** 541-560 30

29 **Security & provenance** 561-580 31

30 **Reuse & packaging** 581-600 32

**Resources & sources** 33-35

29 direct URLs, with references to the catalog categories.

# Task contracts

001-020 / 20 building blocks

001 **outcome-contract**
Describe the result the run must leave.

002 **acceptance-matrix**
Map requirements to observable acceptance checks.

003 **deliverable-manifest**
List required files before starting work.

004 **input-inventory**
Identify the material the task can use.

005 **scope-boundary**
Separate assigned work from adjacent requests.

006 **non-goal-register**
Record explicitly excluded outcomes.

007 **constraint-ledger**
Keep deadlines, formats and exclusions visible.

008 **reader-profile**
Name who will use the resulting artifact.

009 **output-format-contract**
Specify the structure of the final output.

010 **quality-threshold**
Define the minimum acceptable result.

011 **ambiguity-register**
Record questions that change the outcome.

012 **assumption-log**
Keep provisional assumptions distinguishable from facts.

013 **dependency-contract**
Name prerequisites needed before each stage.

014 **priority-order**
Resolve which requirement wins in a conflict.

015 **authorization-scope**
Describe actions the user has actually authorized.

016 **stop-condition**
Specify when the run should stop.

017 **escalation-condition**
Identify situations requiring outside input.

018 **review-owner**
Assign responsibility for accepting the result.

019 **success-evidence-map**
Connect each success claim to supporting evidence.

020 **task-brief-version**
Track changes to the assigned task.

FOUNDATIONS

[R02] Claude Code - Best practices

[R03] Anthropic - Building effective agents

Full URLs: Resources & sources, pp. 33-35.

# Project instructions

021-040 / 20 building blocks

021 **root-claude-md**
Store standing project instructions at the root.

022 **path-scoped-rules**
Apply instructions when relevant paths are accessed.

023 **nested-project-guidance**
Keep package-specific conventions near their code.

024 **user-level-preferences**
Separate personal conventions from team rules.

025 **managed-policy-layer**
Identify organization-controlled configuration.

026 **local-instruction-overlay**
Keep private project preferences outside shared files.

027 **instruction-load-audit**
Inspect which instruction files reached the session.

028 **instruction-precedence-map**
Resolve conflicts between configuration levels.

029 **rule-path-matcher**
Limit scoped guidance to intended file patterns.

030 **standing-fact-filter**
Keep transient task details out of permanent guidance.

031 **instruction-size-budget**
Limit always-loaded guidance to useful content.

032 **concrete-rule-wording**
Write rules with observable actions.

033 **contradiction-audit**
Find incompatible instructions before a long run.

034 **stale-rule-pruning**
Remove guidance that no longer applies.

035 **import-cost-check**
Account for context consumed by imported files.

036 **reference-file-index**
Point to detailed material instead of duplicating it.

037 **source-policy**
Define acceptable sources for factual claims.

038 **workspace-path-policy**
Document where references and outputs belong.

039 **response-style-rule**
Specify durable communication conventions.

040 **context-enforcement-split**
Distinguish written guidance from enforced controls.

FOUNDATIONS

[R04] Claude Code - Project memory

[R05] Claude Code - Settings and precedence

Full URLs: Resources & sources, pp. 33-35.

# Context retrieval

041-060 / 20 building blocks

041 **just-in-time-context**
Retrieve information when the next decision needs it.

042 **progressive-disclosure**
Reveal detail through focused successive reads.

043 **file-glob-discovery**
Find candidate files before opening their contents.

044 **scoped-text-search**
Search only the relevant part of a workspace.

045 **symbol-first-navigation**
Follow definitions and references for code questions.

046 **metadata-first-scan**
Use paths and timestamps to prioritize reads.

047 **reference-pointer-cache**
Retain lightweight identifiers for later retrieval.

048 **query-reformulation**
Refine searches after inspecting the first results.

049 **section-targeted-fetch**
Retrieve the passage that supports the task.

050 **chunk-boundary-design**
Split references without severing useful context.

051 **retrieval-relevance-filter**
Discard material unrelated to the current decision.

052 **retrieval-deduplication**
Avoid loading identical evidence repeatedly.

053 **source-freshness-check**
Check whether retrieved information remains current.

054 **missing-context-detector**
Notice when required evidence has not been retrieved.

055 **context-window-ledger**
Track what occupies the working context.

056 **tool-result-trimming**
Remove redundant output while preserving useful evidence.

057 **hybrid-retrieval-plan**
Combine initial reference selection with later search.

058 **retrieval-failure-note**
Save the identifier and reason for failed retrieval.

059 **context-selection-rationale**
Record why a reference matters to the task.

060 **context-coverage-check**
Confirm required topics have supporting context.

FOUNDATIONS

[R06] Anthropic - Context engineering

[R07] Anthropic - Writing effective tools

Full URLs: Resources & sources, pp. 33-35.

# Memory

061-080 / 20 building blocks

061 **confirmed-decision-store**
Persist decisions approved for later reuse.

062 **preference-confirmation**
Confirm whether a correction should become permanent.

063 **fact-source-link**
Attach evidence to durable factual notes.

064 **fact-checked-date**
Record when a stored fact was verified.

065 **memory-expiry-rule**
Identify facts that need periodic rechecking.

066 **memory-conflict-resolution**
Reconcile incompatible notes before using them.

067 **memory-scope-boundary**
Keep task notes separate from project knowledge.

068 **memory-relevance-gate**
Load only notes relevant to the present task.

069 **working-note-buffer**
Keep temporary reasoning outside permanent guidance.

070 **episodic-run-summary**
Save what happened during one completed run.

071 **procedural-memory**
Record a workflow learned through repeated work.

072 **decision-rationale**
Preserve why a lasting choice was made.

073 **correction-history**
Keep meaningful changes to prior conclusions.

074 **memory-provenance**
Record the origin of each durable note.

075 **memory-index**
Provide concise pointers into longer notes.

076 **memory-review-routine**
Inspect saved preferences before relying on them.

077 **sensitive-note-exclusion**
Avoid retaining secrets in general project notes.

078 **memory-write-permission**
Control where persistent notes can be written.

079 **memory-retrieval-test**
Check that a new session finds needed notes.

080 **forgetting-policy**
Remove obsolete or unsupported retained material.

## FOUNDATIONS

[R04] Claude Code - Project memory

[R06] Anthropic - Context engineering

Full URLs: Resources & sources, pp. 33-35.

# Reusable skills

081-100 / 20 building blocks

081 **skill-trigger-description**
Describe when a reusable procedure fits.

082 **skill-frontmatter**
Declare the procedure's name and configuration.

083 **skill-argument-contract**
Define how task arguments enter the procedure.

084 **skill-input-paths**
Identify the reference files a skill expects.

085 **skill-output-paths**
Name the artifacts a skill must save.

086 **skill-step-results**
Give each procedure step an observable output.

087 **skill-review-step**
Include a defined verification stage.

088 **skill-support-files**
Keep detailed references beside the main instructions.

089 **skill-example-set**
Provide representative examples of expected use.

090 **manual-invocation-mode**
Reserve a procedure for explicit user invocation.

091 **model-invocation-mode**
Allow matching tasks to load a procedure.

092 **skill-tool-scope**
Specify which tools the procedure needs.

093 **forked-skill-context**
Run a suitable procedure in isolated context.

094 **preloaded-worker-skill**
Supply a worker with its required procedure.

095 **skill-effort-setting**
Choose supported reasoning effort for the workflow.

096 **skill-discovery-audit**
Confirm that the intended skill is available.

097 **skill-name-collision**
Resolve duplicate procedure names across scopes.

098 **skill-namespace**
Distinguish procedures supplied by different packages.

099 **skill-regression-case**
Keep examples that caught earlier workflow mistakes.

100 **skill-improvement-loop**
Update the procedure after a recurring correction.

FOUNDATIONS

[R08] Claude Code - Skills

[R09] Claude Code - Extension overview

Full URLs: Resources & sources, pp. 33-35.

# Planning

101-120 / 20 building blocks

101 **plan-before-execution**
Outline work before changing the environment.

102 **task-decomposition**
Split a broad objective into inspectable units.

103 **dependency-graph**
Represent prerequisites between units of work.

104 **critical-path-selection**
Prioritize work that gates the final result.

105 **plan-granularity**
Choose steps small enough to verify.

106 **unknown-first-investigation**
Resolve consequential uncertainty before implementation.

107 **read-before-edit**
Inspect relevant material before modifying it.

108 **prototype-first-pass**
Test an uncertain approach with a small example.

109 **risk-weighted-sequencing**
Address fragile assumptions early in the workflow.

110 **phase-entry-condition**
Define what must hold before a phase starts.

111 **phase-exit-condition**
Define evidence required to leave a phase.

112 **parallelizability-check**
Identify work that can proceed independently.

113 **plan-revision-trigger**
Replan when new evidence changes the approach.

114 **plan-progress-link**
Connect planned steps to saved progress.

115 **implementation-choice-note**
Record consequential choices before they are forgotten.

116 **contingency-branch**
Keep an alternative for a likely failure.

117 **minimal-next-action**
Select one concrete continuation step.

118 **plan-drift-check**
Compare current actions with the accepted objective.

119 **verification-planning**
Choose checks before claiming completion.

120 **acceptance-driven-planning**
Work backward from the required final evidence.

FOUNDATIONS

[R03] Anthropic - Building effective agents

[R10] Claude Code - Common workflows

Full URLs: Resources & sources, pp. 33-35.

# Workflow orchestration

121-140 / 20 building blocks

121 **prompt-chaining**
Pass defined outputs between sequential model steps.

122 **task-routing**
Choose a workflow based on the request.

123 **workflow-dag**
Represent execution as a dependency graph.

124 **fan-out-work**
Dispatch independent bounded work units.

125 **fan-in-results**
Combine completed results into one decision.

126 **map-reduce-analysis**
Analyze partitions before synthesizing the whole.

127 **orchestrator-worker-pattern**
Let one controller assign bounded worker tasks.

128 **evaluator-optimizer-loop**
Revise outputs using structured evaluation feedback.

129 **workflow-state-machine**
Make permitted stage transitions explicit.

130 **conditional-branching**
Choose the next stage from inspected results.

131 **gate-before-mutation**
Check prerequisites before a state-changing step.

132 **stage-artifact-passing**
Transfer saved outputs between workflow stages.

133 **queue-backed-dispatch**
Use a queue to manage pending work.

134 **concurrency-cap**
Limit simultaneously active work units.

135 **backpressure-control**
Slow dispatch when downstream stages are overloaded.

136 **workflow-cancellation**
Stop pending work when the task is cancelled.

137 **barrier-synchronization**
Wait for required inputs before integration.

138 **partial-result-policy**
Decide how incomplete branches affect the result.

139 **workflow-reentry**
Resume at the appropriate saved stage.

140 **branch-result-selection**
Choose outputs using explicit acceptance criteria.

FOUNDATIONS

[R03] Anthropic - Building effective agents

[R09] Claude Code - Extension overview

Full URLs: Resources & sources, pp. 33-35.

# The agent loop

141-160 / 20 building blocks

141 **observe-act-check-loop**
Inspect state, act, then inspect the outcome.

142 **turn-state-snapshot**
Capture relevant state at each turn boundary.

143 **action-selection**
Choose the next tool operation from current evidence.

144 **observation-normalization**
Convert tool responses into usable state updates.

145 **result-before-next-action**
Use the last result before dependent work.

146 **loop-invariant**
Keep a required condition true across turns.

147 **loop-progress-signal**
Identify measurable movement toward the objective.

148 **no-progress-detector**
Notice repeated turns without useful advancement.

149 **repeated-error-detector**
Recognize an unchanged failure condition.

150 **context-refresh-point**
Reload important state when context becomes stale.

151 **interrupt-handling**
Incorporate user steering during active work.

152 **steering-priority**
Apply a correction without losing the main objective.

153 **pending-work-register**
Track actions whose results have not arrived.

154 **loop-termination-reason**
Record why execution stopped.

155 **final-result-assembly**
Combine outputs and checks into the final response.

156 **end-state-inspection**
Open the outcome left in the environment.

157 **action-observation-link**
Associate each observation with its initiating action.

158 **loop-transition-log**
Record changes between runtime states.

159 **error-to-next-step**
Translate a failure into a concrete recovery action.

160 **completion-before-exit**
Check acceptance evidence before ending the loop.

FOUNDATIONS

[R11] Claude Code - How the agent loop works

[R12] Claude Code - Agent SDK overview

Full URLs: Resources & sources, pp. 33-35.

# Tool schemas

161-180 / 20 building blocks

161 **purpose-specific-tool**
Give a tool one clear responsibility.

162 **unambiguous-tool-name**
Choose a name that distinguishes its function.

163 **tool-description-contract**
Explain the operation and when to use it.

164 **input-json-schema**
Describe argument structure in a machine-readable schema.

165 **required-field-design**
Require only values necessary for the operation.

166 **enum-argument**
Constrain an argument to meaningful options.

167 **argument-range-check**
Reject values outside the supported range.

168 **path-argument-semantics**
Clarify how file paths are interpreted.

169 **identifier-argument**
Accept a stable resource identifier.

170 **parameter-defaults**
Make omitted-argument behavior explicit.

171 **output-json-schema**
Define the structure of returned data.

172 **structured-tool-result**
Return fields the next step can inspect.

173 **compact-result-mode**
Offer useful output without excessive context.

174 **paginated-result-contract**
Explain how to retrieve additional result pages.

175 **truncation-indicator**
Mark when a response omits data.

176 **error-code-taxonomy**
Distinguish failure classes in tool responses.

177 **actionable-error-message**
Return information useful for recovery.

178 **usage-example**
Show a representative valid tool call.

179 **tool-capability-boundary**
State the operation's limitations.

180 **overlapping-tool-pruning**
Remove tools with confusingly similar responsibilities.

## FOUNDATIONS

[R07] Anthropic - Writing effective tools

[R13] Model Context Protocol - Tools specification

Full URLs: Resources & sources, pp. 33-35.

# Tool execution

181-200 / 20 building blocks

181 **argument-validation**
Check a proposed call before executing it.

182 **precondition-check**
Verify the environment needed for an operation.

183 **postcondition-check**
Inspect the expected state after an operation.

184 **tool-call-deadline**
Bound how long an operation may run.

185 **retry-classification**
Separate retryable failures from permanent ones.

186 **exponential-backoff**
Increase retry intervals after repeated transient failures.

187 **retry-jitter**
Spread retries to avoid synchronized bursts.

188 **retry-attempt-cap**
Limit attempts for one operation.

189 **idempotency-key**
Identify repeated submissions of the same action.

190 **read-before-retry**
Inspect destination state after an uncertain write.

191 **side-effect-declaration**
Identify which state a tool can change.

192 **exit-code-inspection**
Use process status when judging command success.

193 **stdout-stderr-separation**
Keep normal output distinguishable from diagnostics.

194 **working-directory-binding**
Run commands in the intended workspace.

195 **shell-quoting**
Preserve literal argument content safely.

196 **command-injection-check**
Prevent untrusted values from becoming executable syntax.

197 **process-lifecycle**
Track creation, completion and termination of subprocesses.

198 **background-task-handle**
Retain the identifier for a running operation.

199 **result-size-limit**
Bound tool output retained by the harness.

200 **tool-version-check**
Confirm the installed tool supports the required operation.

FOUNDATIONS

[R07] Anthropic - Writing effective tools

[R12] Claude Code - Agent SDK overview

Full URLs: Resources & sources, pp. 33-35.

# MCP & connectors

201-220 / 20 building blocks

201 **mcp-server-inventory**
List external servers relevant to the workflow.

202 **mcp-transport-choice**
Choose a supported server transport.

203 **connector-auth-check**
Confirm authentication before relying on retrieval.

204 **connector-health-probe**
Test a connection with one known resource.

205 **mcp-tool-discovery**
Inspect the tools exposed by a server.

206 **connector-scope**
Identify which projects can use a connection.

207 **resource-link-binding**
Give retrieval a precise page or resource identifier.

208 **source-section-selection**
Name the part of a remote document needed.

209 **read-write-tool-split**
Distinguish retrieval from state-changing connector actions.

210 **connector-permission-map**
Apply action rules to exposed operations.

211 **unused-server-pruning**
Disable connections unrelated to the current task.

212 **connector-output-contract**
Specify fields needed from remote results.

213 **remote-pagination**
Follow result pages without losing retrieval state.

214 **remote-rate-limit-handling**
Respond to service throttling with bounded retries.

215 **connector-error-context**
Preserve identifiers and errors for failed calls.

216 **remote-state-confirmation**
Inspect the service after an uncertain mutation.

217 **connector-version-pinning**
Track the server version used by a workflow.

218 **mcp-configuration-review**
Inspect connection configuration before reuse.

219 **connector-credential-storage**
Keep authentication material out of task outputs.

220 **connector-smoke-case**
Retain a small known-good retrieval example.

FOUNDATIONS

[R14] Claude Code - MCP connections

[R13] Model Context Protocol - Tools specification

Full URLs: Resources & sources, pp. 33-35.

# Permissions

221-240 / 20 building blocks

221 **least-privilege-tools**
Expose only operations required by the task.

222 **allow-rule**
Permit a defined class of tool actions.

223 **deny-rule**
Reject a defined class of tool actions.

224 **ask-rule**
Require review for a specified operation.

225 **effective-rule-inspection**
Inspect the rules actually loaded by the session.

226 **read-path-policy**
Define which file paths may be read.

227 **write-path-policy**
Define where outputs may be changed.

228 **protected-output-path**
Protect previously approved artifacts from edits.

229 **secret-file-restriction**
Restrict file-tool access to credential files.

230 **command-permission-pattern**
Limit permitted command forms.

231 **external-write-approval**
Review the destination and content of a remote write.

232 **action-preview**
Show the proposed mutation before authorization.

233 **approval-content-binding**
Bind approval to the exact content being changed.

234 **approval-staleness-check**
Recheck authorization when the action changes.

235 **permission-scope-audit**
Distinguish project, user and managed permissions.

236 **permission-inheritance**
Account for inherited rules in the workspace.

237 **permission-test-case**
Exercise a harmless operation against configured rules.

238 **permission-denial-recovery**
Choose permitted alternatives after an operation is denied.

239 **permission-mode-selection**
Choose an appropriate session permission mode.

240 **file-tool-scope-check**
Recognize that file-tool rules are not OS isolation.

FOUNDATIONS

[R15] Claude Code - Permissions

[R05] Claude Code - Settings and precedence

Full URLs: Resources & sources, pp. 33-35.

# Lifecycle hooks

241-260 / 20 building blocks

241 **session-start-hook**
Prepare relevant context when a session begins.

242 **pre-tool-use-hook**
Inspect a tool call before execution.

243 **post-tool-use-hook**
React after a tool operation completes.

244 **tool-failure-hook**
Capture useful information when a call fails.

245 **stop-hook**
Evaluate a condition when the agent attempts to finish.

246 **subagent-start-hook**
Initialize a specialized worker at launch.

247 **subagent-stop-hook**
Inspect a worker's returned result.

248 **notification-hook**
Handle a configured runtime notification.

249 **pre-compact-hook**
Preserve useful state before conversation compaction.

250 **hook-event-matcher**
Limit a handler to relevant events or tools.

251 **hook-input-parser**
Read the runtime event's structured payload.

252 **hook-output-protocol**
Return results in the documented handler format.

253 **hook-exit-semantics**
Use handler exit status as documented.

254 **hook-timeout**
Bound the handler's execution time.

255 **argument-aware-hook**
Judge an operation using its proposed arguments.

256 **state-aware-hook**
Consider saved task state before permitting a call.

257 **format-after-edit-hook**
Run a formatter after relevant changes.

258 **validation-after-edit-hook**
Run a suitable check after a file mutation.

259 **hook-recursion-control**
Prevent repeated triggering from handler side effects.

260 **hook-failure-policy**
Decide how a failed handler affects execution.

FOUNDATIONS

[R16] Claude Code - Hooks reference

[R09] Claude Code - Extension overview

Full URLs: Resources & sources, pp. 33-35.

# Isolation & sandboxes

261-280 / 20 building blocks

261 **filesystem-sandbox**
Restrict process access to intended filesystem areas.

262 **network-sandbox**
Restrict process access to approved network destinations.

263 **writable-root-boundary**
Define the directories a process may change.

264 **readable-root-boundary**
Define the directories a process may inspect.

265 **domain-allowlist**
Permit only needed network hosts.

266 **credential-isolation**
Keep credentials outside the agent's general reach.

267 **process-isolation**
Separate execution environments between tasks.

268 **containerized-runner**
Run work in a bounded container environment.

269 **worktree-isolation**
Use separate checkouts for independent code changes.

270 **temporary-workspace**
Allocate disposable scratch space for a run.

271 **read-only-mount**
Expose references without allowing modification.

272 **restricted-environment-vars**
Avoid inheriting unrelated sensitive environment values.

273 **child-process-policy**
Apply appropriate restrictions to spawned processes.

274 **unsandboxed-retry-review**
Review a request to leave the sandbox.

275 **sandbox-escape-audit**
Inspect operations that bypass normal isolation.

276 **protected-path-test**
Check isolation with a harmless access attempt.

277 **network-denial-diagnostics**
Preserve the denied host and operation.

278 **sandbox-policy-lock**
Prevent unauthorized widening of enforced boundaries.

279 **platform-capability-check**
Confirm which isolation features the host supports.

280 **isolation-cleanup**
Release temporary resources after preserving needed outputs.

FOUNDATIONS

[R17] Claude Code - Sandboxing

[R18] Claude Code - Security

Full URLs: Resources & sources, pp. 33-35.

# Specialized roles

281-300 / 20 building blocks

281 **role-assignment**
Give a worker a bounded responsibility.

282 **role-instruction**
Define the judgment expected from a specialist.

283 **role-tool-allowlist**
Expose tools appropriate to the worker's assignment.

284 **role-context-bundle**
Supply only material the role needs.

285 **role-output-schema**
Define a usable format for the worker's result.

286 **research-worker**
Assign focused source discovery and reading.

287 **implementation-worker**
Assign a concrete scoped change.

288 **verification-worker**
Assign an independent check of saved outputs.

289 **evidence-reviewer**
Connect factual claims to opened primary sources.

290 **security-reviewer**
Inspect applicable action and data boundaries.

291 **performance-reviewer**
Measure relevant runtime or resource behavior.

292 **style-reviewer**
Check a result against approved writing conventions.

293 **architecture-reviewer**
Evaluate structural choices and dependencies.

294 **integration-owner**
Own the merged outcome of specialist work.

295 **read-only-review-role**
Keep a checking worker from mutating the artifact.

296 **role-effort-selection**
Choose supported effort for a specialist task.

297 **role-memory-scope**
Keep a worker's retained notes appropriately scoped.

298 **role-success-criteria**
Specify what counts as a completed assignment.

299 **role-conflict-resolution**
Resolve incompatible specialist findings with evidence.

300 **role-availability-check**
Confirm the configured worker is discoverable.

FOUNDATIONS

[R19] Claude Code - Custom subagents

[R03] Anthropic - Building effective agents

Full URLs: Resources & sources, pp. 33-35.

# Delegation

301-320 / 20 building blocks

301 **bounded-worker-prompt**
Assign one clear task with a defined return.

302 **worker-input-manifest**
List references and artifact paths for a worker.

303 **worker-completion-contract**
Specify evidence a delegated task must return.

304 **independent-review-assignment**
Give verification a separate bounded perspective.

305 **shared-file-ownership**
Assign one writer to each shared artifact.

306 **nonoverlapping-edit-scopes**
Prevent workers from changing the same code area.

307 **worker-worktree**
Give code workers separate checkouts when needed.

308 **worker-status-query**
Inspect whether delegated work is running or complete.

309 **worker-result-wait**
Wait for useful completion events without busy polling.

310 **worker-message-routing**
Deliver findings to the responsible worker.

311 **worker-steering-update**
Send a correction while retaining the assignment.

312 **worker-interruption**
Stop work that is no longer needed.

313 **worker-dependency-link**
Associate delegated work with its prerequisites.

314 **worker-failure-return**
Require a clear blocker and saved partial outputs.

315 **worker-evidence-return**
Return usable sources and verification results.

316 **worker-context-isolation**
Keep intermediate specialist material out of main context.

317 **worker-summary-compression**
Return conclusions with enough supporting detail.

318 **worker-result-integration**
Inspect and merge specialist outputs.

319 **worker-count-budget**
Limit delegation according to task value and resources.

320 **worker-audit-trail**
Record assignments and their returned artifacts.

## FOUNDATIONS

[R19] Claude Code - Custom subagents

[R09] Claude Code - Extension overview

Full URLs: Resources & sources, pp. 33-35.

# Models & reasoning effort

321-340 / 20 building blocks

321 **main-effort-setting**
Choose a supported level for the main session.

322 **reviewer-effort-setting**
Configure effort for bounded verification work.

323 **effort-support-check**
Confirm the model supports the selected setting.

324 **model-capability-match**
Choose a model suitable for the assigned task.

325 **model-version-record**
Record the model used for an experiment.

326 **effort-baseline-run**
Measure one task at the initial effort setting.

327 **effort-ab-comparison**
Compare effort settings on equivalent acceptance criteria.

328 **quality-cost-measurement**
Relate resource use to accepted outcomes.

329 **reasoning-depth-policy**
Choose configured effort by task complexity.

330 **uncertainty-driven-effort**
Increase scrutiny for ambiguous consequential claims.

331 **phase-specific-effort**
Use different settings for drafting and checking.

332 **model-routing-rule**
Route work using task requirements.

333 **fallback-model-policy**
Specify an alternative when the preferred model fails.

334 **model-change-revalidation**
Recheck workflow behavior after a model change.

335 **effort-override-audit**
Inspect settings that override the chosen effort.

336 **reasoning-setting-inspection**
Confirm active configuration before the long task.

337 **review-effort-benchmark**
Measure whether deeper review improves findings.

338 **effort-escalation-trigger**
Define when to reconsider the configured level.

339 **effort-budget-allocation**
Allocate expensive reasoning to valuable decisions.

340 **prose-setting-distinction**
Separate requests to think from real effort settings.

FOUNDATIONS

[R20] Claude Code - Model configuration

[R21] Claude Code - Costs

Full URLs: Resources & sources, pp. 33-35.

# Goals & completion

341-360 / 20 building blocks

341 **completion-condition**
Express when the assigned result is achieved.

342 **goal-evidence-surfacing**
Show results the goal evaluator can inspect.

343 **goal-deliverable-check**
Verify required files exist before reporting success.

344 **goal-verification-report**
Include relevant validation results in conversation.

345 **completion-claim-audit**
Check that success statements match saved artifacts.

346 **unmet-condition-return**
Identify the missing acceptance evidence.

347 **impossible-condition-return**
Explain why a completion condition cannot be met.

348 **goal-clear-operation**
Remove an active goal when instructed.

349 **turn-stop-clause**
Include a model-evaluated turn stopping condition.

350 **hard-limit-separation**
Enforce strict runtime limits outside goal prose.

351 **final-path-return**
Return usable absolute paths to saved outputs.

352 **uncertainty-disclosure**
Identify unresolved parts of the result.

353 **blocker-report**
State the condition preventing completion.

354 **acceptance-checklist-run**
Inspect the required criteria before finishing.

355 **output-open-check**
Open the artifact the user will receive.

356 **completion-state-consistency**
Reconcile progress notes with the final result.

357 **required-work-audit**
Check whether necessary tasks remain unfinished.

358 **review-findings-closure**
Resolve or mark outstanding reviewer findings.

359 **result-evidence-bundle**
Package outputs with supporting checks.

360 **completion-regression-check**
Confirm corrections did not invalidate acceptance.

FOUNDATIONS

[R22] Claude Code - Goals

[R23] Anthropic - Agent evaluations

Full URLs: Resources & sources, pp. 33-35.

# Runtime state & checkpoints

361-380 / 20 building blocks

361 **progress-md**
Maintain a durable task progress note.

362 **current-phase-record**
Record the stage the run has reached.

363 **completed-step-register**
List completed work with associated evidence.

364 **open-issue-register**
Keep unresolved questions visible across turns.

365 **next-action-record**
Save one concrete continuation step.

366 **artifact-state-index**
Identify the current draft and verification files.

367 **feature-status-list**
Track which requested features have passed checks.

368 **checkpoint-before-change**
Save recoverable state before consequential edits.

369 **version-control-snapshot**
Preserve file history through the repository.

370 **state-write-atomicity**
Avoid leaving a partly written state record.

371 **state-schema-version**
Track the structure of persisted state.

372 **state-transition-guard**
Validate required conditions before changing phase.

373 **state-diff**
Inspect how saved state changed during a run.

374 **checkpoint-identifier**
Assign a stable reference to a recovery point.

375 **state-consistency-check**
Compare records with files in the workspace.

376 **state-lock**
Prevent conflicting concurrent state writes.

377 **partial-output-preservation**
Save useful work before a stop or failure.

378 **checkpoint-retention**
Keep the recovery points the workflow may need.

379 **compaction-ready-summary**
Preserve decisions and open work before context resets.

380 **state-provenance-record**
Associate state changes with their cause.

FOUNDATIONS

[R24] Anthropic - Long-running agent harnesses

[R25] Claude Code - Checkpointing

Full URLs: Resources & sources, pp. 33-35.

# Recovery & resuming

381-400 / 20 building blocks

381 **resume-from-progress**
Read the handoff before continuing a task.

382 **fresh-session-recovery**
Check that a new session can reconstruct the work.

383 **last-good-state**
Identify the most recent verified recovery point.

384 **interrupted-run-handoff**
Preserve outputs and next steps after interruption.

385 **failed-stage-restart**
Resume at the failed stage instead of starting over.

386 **replay-safe-operation**
Ensure a repeated operation does not duplicate effects.

387 **uncertain-write-reconciliation**
Inspect whether a timed-out write already completed.

388 **checkpoint-scope-audit**
Know which edits a checkpoint can recover.

389 **shell-change-recovery**
Use explicit recovery for changes outside tracked edits.

390 **subagent-change-recovery**
Preserve recoverable history for delegated mutations.

391 **rollback-plan**
Specify how to undo a failed change.

392 **compensating-action**
Reverse an operation when simple rollback is unavailable.

393 **recovery-smoke-check**
Verify the restored state is usable.

394 **corrupt-state-detection**
Recognize unusable saved progress or checkpoint data.

395 **missing-artifact-recovery**
Find or regenerate a referenced output.

396 **authentication-recovery**
Restore access before repeating a failed retrieval.

397 **environment-repair**
Fix a broken prerequisite before resuming work.

398 **recovery-decision-log**
Record what was restored and why.

399 **resume-evidence-check**
Confirm prior checks still apply after recovery.

400 **recovery-drill**
Test continuation on a harmless interrupted workflow.

FOUNDATIONS

[R24] Anthropic - Long-running agent harnesses

[R25] Claude Code - Checkpointing

Full URLs: Resources & sources, pp. 33-35.

# Validation & tests

401-420 / 20 building blocks

401 **change-relevant-test-selection**
Choose checks that address the actual change.

402 **unit-check**
Validate an isolated behavior or component.

403 **integration-check**
Validate interactions between components.

404 **end-to-end-check**
Exercise the complete user-facing outcome.

405 **smoke-check**
Confirm essential functionality still works.

406 **regression-check**
Recheck behavior affected by a correction.

407 **type-check**
Validate static type constraints where applicable.

408 **lint-check**
Detect applicable structural or style issues.

409 **build-check**
Confirm the artifact can be built.

410 **format-check**
Inspect compliance with the required formatting.

411 **data-schema-check**
Validate saved data against its expected schema.

412 **snapshot-comparison**
Compare a stable output with an accepted reference.

413 **golden-output-case**
Keep an approved result for a known input.

414 **property-based-check**
Test invariants across generated input variations.

415 **metamorphic-check**
Check behavior under meaningful input transformations.

416 **negative-input-case**
Verify handling of invalid or rejected input.

417 **boundary-input-case**
Exercise values at supported limits.

418 **deterministic-fixture**
Use repeatable inputs for reliable validation.

419 **visual-render-check**
Inspect the rendered artifact the user receives.

420 **check-result-preservation**
Save diagnostics supporting the verification claim.

**FOUNDATIONS**

[R02] Claude Code - Best practices

[R23] Anthropic - Agent evaluations

Full URLs: Resources & sources, pp. 33-35.

# Evidence & citations

421-440 / 20 building blocks

421 **claim-inventory**
List factual claims requiring support.

422 **claim-source-map**
Connect each claim to a specific source.

423 **primary-source-selection**
Prefer material published by the responsible authority.

424 **source-open-confirmation**
Inspect the source rather than trusting its title.

425 **quoted-passage-check**
Confirm a quoted passage matches the original.

426 **claim-verdict-table**
Report claims with verdicts and required corrections.

427 **verified-verdict**
Mark a claim supported by inspected evidence.

428 **incorrect-verdict**
Mark a claim contradicted by inspected evidence.

429 **unresolved-verdict**
Mark a claim lacking sufficient evidence.

430 **missing-evidence-note**
Explain what would resolve an uncertain claim.

431 **company-claim-marker**
Identify statements made by the company itself.

432 **inference-marker**
Distinguish an inference from an observed fact.

433 **firsthand-result-boundary**
Use only results the author actually obtained.

434 **publication-date-check**
Record when the supporting material was published.

435 **event-date-check**
Distinguish occurrence dates from publication dates.

436 **version-applicability-check**
Check whether evidence applies to the relevant version.

437 **contradictory-source-review**
Inspect conflicting evidence before choosing wording.

438 **correction-propagation**
Recheck nearby statements after correcting a claim.

439 **reviewed-artifact-binding**
Tie findings to the exact draft reviewed.

440 **citation-target-check**
Ensure a link supports the associated claim.

FOUNDATIONS

[R23] Anthropic - Agent evaluations

[R07] Anthropic - Writing effective tools

Full URLs: Resources & sources, pp. 33-35.

# Evaluation datasets

441-460 / 20 building blocks

441 **representative-task-set**
Include tasks resembling real expected work.

442 **capability-eval-set**
Measure whether a behavior can be performed.

443 **regression-eval-set**
Track failures that were previously corrected.

444 **held-out-task-split**
Keep evaluation examples outside workflow tuning.

445 **eval-input-version**
Version the input material used in a benchmark.

446 **expected-outcome-fixture**
Define the target result for each task.

447 **initial-state-fixture**
Prepare a known environment before a trial.

448 **long-horizon-case**
Include tasks spanning many actions or sessions.

449 **ambiguous-request-case**
Test handling of consequential missing information.

450 **noisy-source-case**
Test selection from irrelevant or redundant material.

451 **conflicting-evidence-case**
Test behavior with incompatible source claims.

452 **empty-retrieval-case**
Test a search returning no useful material.

453 **tool-failure-case**
Include a recoverable failed operation.

454 **permission-denial-case**
Include an action rejected by execution rules.

455 **interruption-resume-case**
Test continuation after a stopped run.

456 **context-reset-case**
Test retention across a context reset.

457 **external-write-case**
Test destination and authorization handling safely.

458 **malicious-content-case**
Test instructions embedded in untrusted material.

459 **eval-coverage-matrix**
Map cases to the behaviors they exercise.

460 **eval-case-provenance**
Record why a benchmark example was included.

FOUNDATIONS

[R23] Anthropic - Agent evaluations

[R07] Anthropic - Writing effective tools

Full URLs: Resources & sources, pp. 33-35.

# Graders & scoring

461-480 / 20 building blocks

461 **exact-match-grader**
Compare output with a required literal value.

462 **schema-grader**
Check structured output against declared constraints.

463 **execution-grader**
Use actual execution to judge the result.

464 **end-state-grader**
Inspect the state left in the environment.

465 **transcript-grader**
Inspect recorded actions and observations.

466 **model-based-grader**
Use a model for bounded qualitative evaluation.

467 **human-review-grader**
Collect expert judgment for suitable criteria.

468 **rubric-based-grader**
Apply explicit dimensions and scoring rules.

469 **pairwise-result-review**
Compare two outcomes on the same task.

470 **multi-grader-bundle**
Combine complementary checking methods.

471 **partial-credit-policy**
Score useful incomplete outcomes consistently.

472 **weighted-criterion-score**
Weight dimensions according to their importance.

473 **false-positive-audit**
Find incorrect success judgments.

474 **false-negative-audit**
Find valid results rejected by a grader.

475 **grader-calibration-set**
Compare judgments against agreed reference examples.

476 **grader-disagreement-review**
Inspect cases where evaluators differ.

477 **grader-version-record**
Track the evaluation logic used for a run.

478 **blind-result-review**
Hide irrelevant run identity during judgment.

479 **threshold-sensitivity-check**
Inspect behavior around acceptance thresholds.

480 **grading-evidence-retention**
Preserve the observations supporting each verdict.

FOUNDATIONS

[R23] Anthropic - Agent evaluations

[R26] Claude Code - Structured agent outputs

Full URLs: Resources & sources, pp. 33-35.

# Observability

481-500 / 20 building blocks

481 **run-identifier**
Associate logs and artifacts with one execution.

482 **turn-identifier**
Identify a specific interaction within the run.

483 **trace-correlation**
Connect related operations across components.

484 **tool-call-span**
Record the boundaries of a tool operation.

485 **stage-latency**
Measure time spent in each workflow stage.

486 **error-event**
Record a failure with useful diagnostic context.

487 **retry-counter**
Count repeated attempts for an operation.

488 **token-usage-metric**
Track reported model token use.

489 **model-call-count**
Count model invocations in the complete workflow.

490 **worker-usage-attribution**
Include delegated work in resource accounting.

491 **structured-event-log**
Record machine-readable runtime events.

492 **log-payload-redaction**
Remove sensitive content from retained diagnostics.

493 **trace-sampling-policy**
Choose which execution details to retain.

494 **metric-cardinality-control**
Keep metric labels from expanding without bound.

495 **progress-event**
Emit a meaningful state advancement.

496 **stall-alert**
Detect a run that stopped making progress.

497 **run-timeline**
Present major operations in temporal order.

498 **failure-cluster-analysis**
Group related failures for workflow improvement.

499 **telemetry-export-check**
Confirm configured signals reach their destination.

500 **observability-retention-policy**
Keep diagnostics for an appropriate period.

FOUNDATIONS

[R27] Claude Code - Usage monitoring

[R28] OpenTelemetry - Traces

Full URLs: Resources & sources, pp. 33-35.

# Resources & budgets

501-520 / 20 building blocks

501 **run-spend-ceiling**
Use execution controls for a hard spending limit.

502 **wall-clock-deadline**
Bound total elapsed execution time externally.

503 **turn-count-accounting**
Track turns consumed by a run.

504 **per-call-token-budget**
Allocate model output capacity for one call.

505 **context-occupancy-budget**
Limit retained material in working context.

506 **retry-resource-budget**
Include retries in the run's total allowance.

507 **worker-resource-budget**
Reserve resources for delegated work.

508 **review-resource-reserve**
Leave enough capacity for final verification.

509 **rate-limit-budget**
Respect service request allowances.

510 **concurrency-resource-budget**
Control resource use from simultaneous tasks.

511 **latency-target**
Set a measured responsiveness objective.

512 **cache-hit-accounting**
Inspect which repeated inputs benefit from caching.

513 **redundant-call-pruning**
Remove repeated calls that add no evidence.

514 **result-reuse-policy**
Reuse valid prior outputs when inputs match.

515 **resource-warning-threshold**
Detect approaching execution limits.

516 **resource-stop-handler**
Preserve progress when a hard limit is reached.

517 **accepted-output-cost**
Measure resources per accepted result.

518 **repair-cost-accounting**
Include human and agent correction effort.

519 **controlled-cost-experiment**
Change one component while keeping the task fixed.

520 **resource-report**
Return meaningful usage information after a run.

FOUNDATIONS

[R21] Claude Code - Costs

[R29] Claude Code - Programmatic execution

Full URLs: Resources & sources, pp. 33-35.

# Environment & bootstrap

521-540 / 20 building blocks

521 **workspace-bootstrap**
Prepare the directories required by the workflow.

522 **source-folder**
Keep reference material in a known location.

523 **draft-folder**
Keep working outputs separate from accepted files.

524 **approved-output-folder**
Identify files already accepted for use.

525 **runtime-version-pin**
Record the interpreter or runtime version.

526 **dependency-lockfile**
Preserve resolved dependency versions.

527 **dependency-availability-check**
Confirm required libraries are installed.

528 **working-tree-inspection**
Inspect local changes before beginning code work.

529 **branch-selection**
Choose the intended revision for a task.

530 **initialization-script**
Prepare repeatable prerequisites for a session.

531 **bootstrap-smoke-run**
Confirm the prepared environment can perform basic work.

532 **configuration-load-check**
Inspect the effective runtime configuration.

533 **credential-presence-check**
Confirm required authentication exists without exposing it.

534 **network-availability-check**
Verify access to required services.

535 **source-access-check**
Open a known reference before the full task.

536 **output-path-preflight**
Confirm the intended output location is writable.

537 **locale-encoding-check**
Preserve the required language and character encoding.

538 **platform-path-handling**
Use paths appropriate to the execution host.

539 **reproducible-environment**
Keep enough configuration to repeat the run.

540 **environment-fingerprint**
Record the relevant environment used for verification.

**FOUNDATIONS**

[R10] Claude Code - Common workflows

[R24] Anthropic - Long-running agent harnesses

Full URLs: Resources & sources, pp. 33-35.

# Deliverables & artifacts

541-560 / 20 building blocks

541 **artifact-manifest**
List the outputs produced by a run.

542 **artifact-naming-rule**
Use filenames that identify purpose and version.

543 **absolute-output-link**
Return a path the user can open.

544 **source-and-render-pair**
Save editable source alongside the rendered result.

545 **draft-review-pair**
Save the draft together with its checking report.

546 **artifact-checksum**
Identify exact file contents with a digest.

547 **artifact-format-validation**
Check the file matches its advertised format.

548 **artifact-openability**
Confirm the delivered file opens successfully.

549 **render-preview**
Inspect a visual preview of the artifact.

550 **export-dimension-check**
Verify requested dimensions in the export.

551 **text-clipping-check**
Inspect whether all rendered text is visible.

552 **encoding-integrity-check**
Check the final encoded file preserves the content.

553 **artifact-version-label**
Distinguish revisions of the same deliverable.

554 **artifact-change-summary**
Describe consequential changes between revisions.

555 **artifact-acceptance-status**
Mark whether an output is draft or accepted.

556 **artifact-source-citation**
Keep supporting sources with the deliverable.

557 **artifact-portability-check**
Confirm the output works outside its creation context.

558 **artifact-bundle**
Package related files in a coherent handoff.

559 **artifact-recovery-copy**
Preserve a useful prior version before replacement.

560 **artifact-delivery-confirmation**
Ensure the final response points to the actual file.

FOUNDATIONS

[R26] Claude Code - Structured agent outputs

[R02] Claude Code - Best practices

Full URLs: Resources & sources, pp. 33-35.

# Security & provenance

561-580 / 20 building blocks

561 **instruction-data-boundary**
Treat retrieved text as evidence rather than authority.

562 **untrusted-content-label**
Identify material from outside the trusted request.

563 **prompt-injection-review**
Inspect attempts to redirect the agent through data.

564 **source-provenance-chain**
Trace evidence to its original publisher.

565 **remote-content-verification**
Check material before incorporating it into work.

566 **secret-redaction**
Remove credentials from outputs and diagnostics.

567 **credential-scope-minimization**
Use authentication limited to required services.

568 **outbound-data-review**
Inspect what information an external action sends.

569 **external-destination-check**
Verify the target of a remote write.

570 **symlink-target-check**
Resolve links before sensitive filesystem operations.

571 **path-traversal-check**
Keep computed paths inside intended boundaries.

572 **destructive-action-guard**
Review operations that irreversibly remove state.

573 **user-authorization-record**
Preserve the scope of approved consequential actions.

574 **untrusted-code-execution-gate**
Review foreign code before running it.

575 **dependency-integrity-check**
Inspect the provenance of executable dependencies.

576 **least-data-exposure**
Give tools only the information needed.

577 **trust-boundary-map**
Identify where data crosses authority boundaries.

578 **security-failure-fixture**
Test an applicable boundary with harmless inputs.

579 **incident-evidence-preservation**
Keep useful diagnostics after a security failure.

580 **security-policy-version**
Track the rules that applied to an execution.

FOUNDATIONS

[R18] Claude Code - Security

[R17] Claude Code - Sandboxing

Full URLs: Resources & sources, pp. 33-35.

# Reuse & packaging

581-600 / 20 building blocks

581 **harness-template**
Package a workflow for repeated tasks.

582 **configuration-version**
Track changes to reusable runtime settings.

583 **configuration-diff-review**
Inspect the effect of a settings change.

584 **config-schema-validation**
Check configuration against supported structure.

585 **config-precedence-test**
Confirm which settings win in a conflict.

586 **workflow-parameterization**
Supply new material without rewriting the procedure.

587 **reference-output-set**
Keep accepted examples for future comparison.

588 **reusable-review-format**
Use a consistent evidence report across tasks.

589 **component-compatibility-check**
Confirm combined extensions work together.

590 **plugin-package**
Bundle related extension components for reuse.

591 **plugin-namespace-policy**
Avoid collisions between packaged components.

592 **plugin-version-pin**
Record the extension version used by a run.

593 **release-regression-suite**
Check important cases before shipping a harness change.

594 **workflow-migration-note**
Explain how to move existing tasks to a revision.

595 **deprecation-plan**
Retire obsolete components without losing needed behavior.

596 **harness-change-log**
Record meaningful workflow improvements.

597 **controlled-rollout**
Introduce a changed setup to a bounded task set.

598 **rollback-configuration**
Keep a usable prior setup for recovery.

599 **recurring-failure-fix**
Improve the reusable process after repeated mistakes.

600 **accepted-run-benchmark**
Compare changes using completed inspectable work.

FOUNDATIONS

[R09] Claude Code - Extension overview

[R05] Claude Code - Settings and precedence

Full URLs: Resources & sources, pp. 33-35.

# Resources & sources

Primary references for the concepts in this catalog. The 600 labels and explanations are authored for the collection; they are not a list copied from these sources.

R01 **@beamnxw - Original article**

https://x.com/beamnxw/status/2107522996046905797

Original article and theme for the collection.

R02 **Claude Code - Best practices**

https://code.claude.com/docs/en/best-practices

Used in: 01 Task contracts, 21 Validation & tests, 28 Deliverables & artifacts.

R03 **Anthropic - Building effective agents**

https://www.anthropic.com/engineering/building-effective-agents

Used in: 01 Task contracts, 06 Planning, 07 Workflow orchestration, 15 Specialized roles.

R04 **Claude Code - Project memory**

https://code.claude.com/docs/en/memory

Used in: 02 Project instructions, 04 Memory.

R05 **Claude Code - Settings and precedence**

https://code.claude.com/docs/en/settings

Used in: 02 Project instructions, 12 Permissions, 30 Reuse & packaging.

R06 **Anthropic - Context engineering**

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Used in: 03 Context retrieval, 04 Memory.

R07 **Anthropic - Writing effective tools**

https://www.anthropic.com/engineering/writing-tools-for-agents

Used in: 03 Context retrieval, 09 Tool schemas, 10 Tool execution, 22 Evidence & citations, 23 Evaluation datasets.

R08 **Claude Code - Skills**

https://code.claude.com/docs/en/skills

Used in: 05 Reusable skills.

R09 **Claude Code - Extension overview**

https://code.claude.com/docs/en/features-overview

Used in: 05 Reusable skills, 07 Workflow orchestration, 13 Lifecycle hooks, 16 Delegation, 30 Reuse & packaging.

R10 **Claude Code - Common workflows**

https://code.claude.com/docs/en/common-workflows

Used in: 06 Planning, 27 Environment & bootstrap.

# Resources & sources

Primary references for the concepts in this catalog. The 600 labels and explanations are authored for the collection; they are not a list copied from these sources.

R11 **Claude Code - How the agent loop works**
https://code.claude.com/docs/en/how-claude-code-works
Used in: 08 The agent loop.

R12 **Claude Code - Agent SDK overview**
https://code.claude.com/docs/en/agent-sdk/overview
Used in: 08 The agent loop, 10 Tool execution.

R13 **Model Context Protocol - Tools specification**
https://modelcontextprotocol.io/specification/2025-11-25/server/tools
Used in: 09 Tool schemas, 11 MCP & connectors.

R14 **Claude Code - MCP connections**
https://code.claude.com/docs/en/mcp
Used in: 11 MCP & connectors.

R15 **Claude Code - Permissions**
https://code.claude.com/docs/en/permissions
Used in: 12 Permissions.

R16 **Claude Code - Hooks reference**
https://code.claude.com/docs/en/hooks
Used in: 13 Lifecycle hooks.

R17 **Claude Code - Sandboxing**
https://code.claude.com/docs/en/sandboxing
Used in: 14 Isolation & sandboxes, 29 Security & provenance.

R18 **Claude Code - Security**
https://code.claude.com/docs/en/security
Used in: 14 Isolation & sandboxes, 29 Security & provenance.

R19 **Claude Code - Custom subagents**
https://code.claude.com/docs/en/sub-agents
Used in: 15 Specialized roles, 16 Delegation.

R20 **Claude Code - Model configuration**
https://code.claude.com/docs/en/model-config
Used in: 17 Models & reasoning effort.

# Resources & sources

Primary references for the concepts in this catalog. The 600 labels and explanations are authored for the collection; they are not a list copied from these sources.

R21 **Claude Code - Costs**

https://code.claude.com/docs/en/costs

Used in: 17 Models & reasoning effort, 26 Resources & budgets.

R22 **Claude Code - Goals**

https://code.claude.com/docs/en/goal

Used in: 18 Goals & completion.

R23 **Anthropic - Agent evaluations**

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Used in: 18 Goals & completion, 21 Validation & tests, 22 Evidence & citations, 23 Evaluation datasets, 24 Graders & scoring.

R24 **Anthropic - Long-running agent harnesses**

https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

Used in: 19 Runtime state & checkpoints, 20 Recovery & resuming, 27 Environment & bootstrap.

R25 **Claude Code - Checkpointing**

https://code.claude.com/docs/en/checkpointing

Used in: 19 Runtime state & checkpoints, 20 Recovery & resuming.

R26 **Claude Code - Structured agent outputs**

https://code.claude.com/docs/en/agent-sdk/structured-outputs

Used in: 24 Graders & scoring, 28 Deliverables & artifacts.

R27 **Claude Code - Usage monitoring**

https://code.claude.com/docs/en/monitoring-usage

Used in: 25 Observability.

R28 **OpenTelemetry - Traces**

https://opentelemetry.io/docs/concepts/signals/traces/

Used in: 25 Observability.

R29 **Claude Code - Programmatic execution**

https://code.claude.com/docs/en/headless

Used in: 26 Resources & budgets.